

Reviewer 1

Novelty: 6 (reasonably novel - there are some notable differences from existing ideas and probably enough to turn into a new paper)

Rationale: The construction of Temporal Graph sounds novel. The research question is also relatively underexplored, but necessary for coding in domains like distributed systems.

Feasibility: 6 (Feasible: Can be executed within the given constraints with some reasonable planning.)

Rationale: The data collection part should be the most challenging part. Collecting high-quality coding problems that involve complex temporal dependencies could be hard. Also, the human evaluation might also take time to execute.

Expected Effectiveness: 6 (Somewhat effective: There is a decent chance that the proposed idea can beat existing baselines by moderate margins on a few benchmarks.)

Rationale: With specific prompting techniques, the proposed method should outperform baselines in terms of temporal dependencies.

Excitement: 7

Rationale: I think this should be more exciting than most of the borderline papers since we are working on a new problem. The collected data should also be super useful.

Overall Score: 7 (Good idea, would be accepted by major AI conferences)

Rationale: Again, working on a novel problem makes it better than most of the prompting papers.

Confidence: 4 (You are confident but not absolutely certain that the evaluation is correct)

Reviewer 2

Novelty: 5 (somewhat novel - there are differences from existing ideas but not enough to turn into a new paper)

Rationale: Although I am not entirely familiar with the field of generating temporally adaptive programs, I suspect some similar ideas can be found in software engineering works (e.g., ICSE). More concretely on the method, it is rather similar to code generation with intermediate state reasoning, which has been explored in several multi-step, conversational code generation works, e.g:

1. Zheng, Tianyu, et al. "Opencodeinterpreter: Integrating code generation with execution and refinement."
2. Cao, Liuwen, et al. "Beyond Code: Evaluate Thought Steps for Complex Code Generation." Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024). 2024.
3. Nijkamp, Erik, et al. "Codegen: An open large language model for code with multi-turn program synthesis."

Feasibility: 3 (Very challenging: there are flaws in the proposed method or experiments, or the experiments require compute/human resources beyond any academic lab)

Rationale: It would be pretty hard to collect such datasets (e.g., would mostly require a whole repository), further, it would be difficult to generate executable test cases to verify the multiple problems created. Especially because the task targets temporally-dependent modules in the program, it may necessitate domain experts to carefully construct examples and tests, which would demand a lot of time and costs.

Expected Effectiveness: 5 (Somewhat ineffective: There might be some chance that the proposed idea can work better than existing baselines but the improvement will be marginal or inconsistent.)

Rationale: I am not very confident that the model can solve this complex temporally-dependent programming problems with reasonable correctness. Furthermore, because the current method is basically prompting, which may have a very low performance upper bound. Therefore, I don't expect the proposed method to improve significantly on code generation.

Excitement: 4

Rationale: Overall, I don't expect this method to bring substantial improvements, hence am less excited about the potential of this method. It would still be an interesting problem to solve, particularly in bringing more challenging coding problems and proposed corresponding methods. With this being said, given the current performance of models, building a solid benchmark regarding this temporal code generation problem may be more exciting than proposing a method that is expectedly not working.

Overall Score: 4 (Ok but not good enough, rejection for major AI conferences)

Rationale: The task of temporal code generation is not the most urgent issue of current code generation models, and the proposed method is expected to not bring much improvement. The method needs to be further refined and go beyond simple prompting to convince the audience of the potential of this thread of methods.

Confidence: 3 (You are fairly confident that the evaluation is correct)

Reviewer 3

Novelty: 10 (very novel - very different from all existing ideas in a very interesting and clever way)

Rationale: This idea studies a very novel problem in LLM-based code generation. Temporal dependencies in code generation should be specifically studied in the era of LLMs.

Feasibility: 5 (Moderately feasible: It can probably be executed within the given time frame but would require careful planning, efficient use of APIs or some advanced computational strategies to overcome the limited GPU resources, and would require some modifications to the original proposal to make it work.)

Rationale: Constructing a reasonable dataset is challenging within a short time. Also, human evaluation might take more time. Whether LLM can construct high-quality graphs in this case is also to be examined.

Expected Effectiveness: 6 (Somewhat effective: There is a decent chance that the proposed idea can beat existing baselines by moderate margins on a few benchmarks.)

Rationale: One needs to build reasonable metrics to show effectiveness. Also, one might need to tune prompts carefully to construct high-quality graphs in this case.

Excitement: 8 (Exciting: would deepen the community's understanding or make major progress in this research direction)

Rationale: This is novel and could have a huge impact on those code generation cases requiring temporal dependencies. But one needs to justify why such use cases are important, and why temporal dependency is the core problem in such use cases.

Overall Score: 9 (Top 15% of all published ideas on this topic at major AI conferences, strong accept)

Rationale: Considering its novelty, valuable dataset, and comprehensiveness of experiment and evaluation design, this could be an impactful work. But one needs to make experiment results concrete by re-examining whether each step works well in practice.

Confidence: 4 (You are confident but not absolutely certain that the evaluation is correct)

X Identities of Example Ideas

We reveal whether each example idea is AI-generated or human-written:

- Human ideas: Example [P](#), Example [R](#), Example [T](#), Example [V](#)
- AI ideas: Example [Q](#), Example [S](#), Example [U](#), Example [W](#)

Y Attempt on Idea Execution Agent

For our execution agent, the input is the generate idea (the full project proposal), and the output is a Python file that can be executed with our specified command. Since there is often a common pipeline of implementing prompting-based research ideas, we provide a manually crafted code file example as template. We attach the full template below:

```
1 import random
2 from tqdm import tqdm
3 from utils import call_api, load_model
4 import random
5 random.seed(2024)
6
7 ## Step 1: Generate synthetic test examples
8 def generate_testset():
9     test_data = [
10         {
11             "input": "Natalia sold clips to 48 of her friends in April, and then
12                 she sold half as many clips in May. How many clips did Natalia sell
13                 altogether in April and May?",
14             "output": "Natalia sold  $48/2 = 24$  clips in May. Natalia sold
15                  $48+24 = 72$  clips altogether in April and May. #### 72"
16         },
17         {
18             "input": "Weng earns $12 an hour for babysitting. Yesterday, she just
19                 did 50 minutes of babysitting. How much did she earn?",
20             "output": "Weng earns  $12/60 = 0.2$  per minute. Working 50
21                 minutes, she earned  $0.2 \times 50 = 10$ . #### 10"
22         },
23         {
24             "input": "Tim has 30 less apples than Martha, and Harry has half as
25                 many apples as Tim. If Martha has 68 apples, how many apples does
26                 Harry have?",
27             "output": "Tim has  $68-30 = 38$  apples. Harry has  $38/2 = 19$ 
28                 apples. #### 19"
29         },
30         {
31             "input": "Four people lost a total of 103 kilograms of weight. The
32                 first person lost 27 kilograms. The second person lost 7 kilograms
33                 less than the first person. The two remaining people lost the same
34                 amount. How many kilograms did each of the last two people lose?",
35             "output": "Second person =  $27 - 7 = 20$  kg  $103 - 27 - 20 = 56$  kg
36                  $56/2 = 28$  kg The last two people
37                 each lost 28 kilograms of weight. #### 28"
38         }
39     ]
40
41     return test_data
42
43 ## Step 2: Implement the baseline method
44 def baseline_method(client, model_name, seed, question):
45     ## zero-shot chain-of-thought
46     prompt = "Answer the following question: {}\n".format(question)
47     prompt += "Think step by step."
48     prompt_messages = [{"role": "user", "content": prompt}]
49     response, _ = call_api(client, model_name, prompt_messages, temperature=0.,
50                             max_tokens=2000, seed=seed, json_output=False)
51     return response.strip()
```